

Security

Read this before connecting Open Hope Beacon to anything real. It is short on purpose, and it tries to be honest rather than reassuring.

Reporting a vulnerability

Do not open a public issue. Use GitHub's private [Security Advisories](https://github.com/Klydo131/Open-Hope-Beacon/security/advisories/new) (<https://github.com/Klydo131/Open-Hope-Beacon/security/advisories/new>) on this repository, which is visible only to maintainers until a fix is published.

Include what an attacker can do stated as an outcome ("any visitor can read X"), the smallest steps that reproduce it, and the commit you tested. You will get an acknowledgement within **7 days**. There is no bounty programme — this is a small project maintained in spare time, and we would rather say so than imply a response time nobody can hold to.

What this app is, in security terms

As shipped, there is nothing to breach. No backend, no accounts, no network calls, no analytics. Every screen runs from a store in the browser, and the sample church is fiction. That is why you can hand it to anybody to try.

Three tests hold that promise rather than a paragraph in a README:

Check	What it refuses
<code>tests/no-backend.js</code>	A database dependency, an API route that stores data, any analytics or error-reporting SDK, any call to an external server.
<code>tests/no-secrets.js</code>	A credential of recognisable shape in any tracked file, a real <code>.env</code> , anything secret exposed under a browser-visible prefix, sample data at a domain that could reach a real inbox.
<code>tests/security-invariants.mjs</code>	A weakened Content-Security-Policy, a URL guard that stops blocking <code>javascript:</code> , CI that could be hijacked by a pull request.

Run them with `npm test`.

The moment that changes: connecting a backend

Everything above stops being the whole picture the day you point this at a real database. From then on **you own the security of what you connected**, and this is the part people get wrong.

Put authorisation in the database, not in the app

The screens in this app decide what to *show*. They cannot decide what somebody is *allowed to have*, because anybody can send a request without using your screens at all.

If your backend supports row-level rules, use them. A rule that lives with the data applies to every route into it — including a screen somebody adds next year without reading this file.

Never put a secret in this repository

Anything the browser can read, every visitor can read. That includes any key in a

backend adapter you write in `lib/backend/`.

Keys belong on a server you control. The browser talks to your server; your server holds the key. `tests/no-secrets.js` fails the build if a credential appears here, but it can only catch shapes it recognises — it is a safety net, not permission to be careless.

Rate limit on your server

Anything enforced in the browser can be skipped by opening the network tab. If an endpoint you add sends email, writes rows, or costs money, limit it server-side, key the limit on something the caller cannot change for free, and give anything that sends messages its own separate ceiling.

Decide what your leaders can see

This app is built so that leaders see counts and Guides see the relationship — a pastor cannot read a Guide's conversations, because there is no screen that shows them. If you connect a backend, that boundary is now yours to enforce in your data rules. It is easy to lose by accident and hard to explain afterwards.

What is not protected, plainly

- **An unlocked device that is signed in.** Whoever holds it sees what its owner

sees. True of all software; worth saying because it is the most likely real-world exposure.

- **Anyone with a legitimate account.** Rules restrict what a role can retrieve.

They cannot stop somebody reading their own records and repeating them.

- **Content you choose to share.** If an administrator uploads a sensitive document to the shared library, the app will faithfully share it.

- **Your hosting provider.** Whoever runs your server and database can see what is on it.
-

Search engines

Your deployment is invisible to search by default, and a church should leave it that way. A church Beacon holds real people's names and conversations, and a shared deep link that gets indexed is the cheapest possible leak — nobody has to break anything, they only have to search.

Three signals say so together: a `<meta name="robots">` tag on every page,

variable, so they cannot end up disagreeing with each other:

```
BEACON_PUBLIC_SITE=1 # opt IN to being findable. Unset, the default, means no.
```

Set it only on a deployment with no real people in it — a public demo or a showcase, where being unfindable is the bug. Do not set it on a church's Beacon.

What this is not: `robots` directives are a request. The large search engines honour them and anything that does not care ignores them. They keep your app out of Google; they are not access control. If a page must not be read by a stranger, it needs a sign-in, not a header.

Sample data

The sample church is fiction and must stay fiction. Names use reserved domains (`.example` , `.test`) that can never resolve to a real inbox, and a test enforces it. **Never commit real people's details**, not even briefly, not even in a branch — a public repository is indexed within minutes and git remembers.

Supported versions
